Neurocomputing 529 (2023) 23–32

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Data imputation in IoT using Spatio-Temporal Variational Auto-Encoder

Shuo Zhang [a], Jinyi Chen [a], Jiayuan Chen [a], Xiaofei Chen [a], Hejiao Huang [a,b,*]

[a] Harbin Institute of Technology, Shen Zhen, China
[b] Guangdong Provincial Key Laboratory of Novel Security Intelligence Technologies, China

ARTICLE INFO

Article history:
Received 29 April 2022
Revised 1 September 2022
Accepted 8 January 2023
Available online 16 January 2023
Communicated by Zidong Wang

Keywords:
Data imputation
IoT
VAE
GCN
GRU

ABSTRACT

In most Internet of Things (IoT) scenes, data missing can be unavoidable when huge number of smart devices are collecting data uninterruptedly. Therefore, data imputation can be an integral part of pre-processing before data mining. It is widely known that IoT time series show strong dependencies in both spatial and temporal dimension, and the spatial relation among the devices is in non-euclidean space. However, most machine-learning-based and deep-learning-based approaches either only take temporal features into account or only catch spatial features in euclidean space. In this paper, we propose a novel network as ST-VAE (Spatio-Temporal Variational Auto-Encoder) to address the problem above. Our architecture is mainly based on Variational Auto-Encoder (VAE). Specifically, two kinds of VAE are utilized. One is for calculating the adjacent matrix of device network which is the essential input of GCN, and the other is for data imputation task based on the spatial and temporal dependencies. Experiments conducted on different real-world and public datasets demonstrate that our ST-VAE can not only populate the missing spatio-temporal data accurately but also outperforms other state-of-art approaches from the whole.

1. Introduction

Along with the climax of Industry 4.0, more and more researches have been conducted on Internet of Things (IoT). Two major parts can be concluded in mainstream IoT architecture: communication network and devices. In most cases, the external environment of IoT can be harsh, such as unfavorable temperature, strong sunshine, high humidity, strong wind, etc. [1] Under this circumstance, data missing occurs for mainly-two reasons: On the one hand, the majority of network in IoT is wireless, which means the signal strength of which can be influenced greatly by the environment. As a result, data transmitted by it may be lost. On the other hand, modern devices of IoT can be complicated and smart as a variety of sensors embedded in them. Harsh environment may accelerate the aging of sensors and impact the continuity of data collection. Taking Intelligent Transportation System (ITS), a typical IoT scene, as an example. According to [29], [30] and [31], traffic data missing rates in the United Sates and China are 15 % and 10 %, and even more than 50 % in Canada. However, ignorance of missing value may have huge impact on the overall performance of ITS, since algorithms of many downstream tasks (e.g., congestion prediction [32] and traffic state classification [33]) need the traffic data to be continuous and completed [37]. When traffic data that with missing value are input, the effectiveness of these algorithms will be greatly reduced, and some even cannot normally run [34]. Therefore, it is significant to populate the missing value before the following analysis work in IoT, not only for the normal operation of the whole system, but also for the decision-making mined from these data.

Data gathered in IoT show two strong features: temporal feature and spatial feature. From the perspective of sensor, data collected by a single sensor are usually organized and stored in chronological order, thus forming a time series. While in terms of device, there might be some spatial relations among device group, whose topological structure is network. Therefore, sensors embedded in them might collect time series that show strong non-euclidean spatial dependencies, and these time series form spatio-temporal data. Taking rainfall collected data in geological disaster as an example: Devices randomly deployed in the same mountain or forest are organized as a group with non-euclidean spatial network topologies, and it is reasonable to impute the missing value according to both the history collected data from the same device (temporal feature) and the current collected data from the spatial correlated devices (non-euclidean spatial feature).

To tackle the challenges above, we propose ST-VAE (Spatio-Temporal Variational Auto-Encoder), a VAE based stochastic approach for populating the missing spatio-temporal data for IoT.

* Corresponding author at: Harbin Institute of Technology, Shen Zhen, China.
E-mail address: huanghejiao@hit.edu.cn (H. Huang).

https://doi.org/10.1016/j.neucom.2023.01.022
0925-2312/

In our network, two kinds of VAE are used for different tasks: AM-VAE (Adjacent Matrix VAE) and DI-VAE (Data Imputation VAE). In detail, we make the following technical contributions:

1. We employ Graph Convolutional Network (GCN) based approach to extract the non-euclidean spatial feature of spatio-temoral data in IoT, in which adjacent matrix is the essential input. However, it is difficult to get the accurate priori adjacent matrix information corresponding to the spatial relation of devices in IoT. Therefore, we innovate VAE to deal with this problem in AM-VAE block, which to the best of our knowledge, is the first one using VAE building adjacent matrix for device network in IoT.
2. Based on the spatial features extracted from GCN, we design a Multi-head Gated Recurrent Unit (GRU) based VAE, called DI-VAE, to catch the temporal features as well as performing the imputation task. It should be noticed that not only the hidden state h of GRU but also the sampling of latent variable z of VAE is time ordered, making it sufficient for DI-VAE to extract temporal dependencies. What's more, we utilize Linear Normalizing Flow to map the Gaussian distribution of z into stochastic space, which adds more robustness to our network.
3. With the combination of AM-VAE and DI-VAE, our stochastic based method can not only perform the data-filling task according to universal spatial and temporal dependencies, but also show excellently generative generalization ability for individual randomness, which make the imputed value more realistic.
4. Sufficient ablation experiments and comparison experiments are conducted to illustrate the effectiveness of our major techniques as well as showing that ST-VAE outperforms many other state-of-art methods.

2. Related work

2.1. Traditional method

Spatio-temporal data imputation has been intensively researched for many years, and traditional methods can mainly be divided into four groups: prediction-based approach, interpolation-based approach, statistic-based approach and decomposition-based approach. Prediction-based approach tends to build a connection between historical and future missing value. Autoregressive integrated moving average (ARIMA), one of the most popular prediction models for time series, is utilized in [2] to the traffic counts imputation for highway agencies. However, the shortcoming of this method is obvious that the information following the missing data is not employed during the imputation, which makes it less effective. Interpolation-based approach fills the missing value with the neighboring value. As for spatio-temporal data, neighboring value can either be the temporal-liking value or spatial-liking value. [3] proposes a data-driven imputation method for sections of road based on their spatial and temporal correlation using a modified K-Nearest Neighbor method. [4] compares Local Least Squares (LLS) with K-Nearest when dealing with missing traffic data imputation. Both the research mentioned above use interpolation-based approaches. Nevertheless, this kind of method is determinative and cannot extract stochastic variations in spatio-temporal data. **Statistic**-based approach learns the statistical rule of the data directly, and undertakes the imputation task based on the rule. The most widely-used method is Principle Component Analysis (PCA), and many improvements are conducted on it. [5] employs PPCA to conduct probabilistic interpretation on PCA when dealing with traffic flow value imputation, while [6] adds Bayesian network to PCA to solve the same problem, which is called BPCA. [7] uses KPCA as imputing methods without any temporal or spatial dependency, and replaces linear kernel with non-linear kernel in PCA. However, this kind of approach focuses too much on constructing probability distribution model, thus losing detailed information varying in different situations. **Decomposition**-based approach intends to decompose origin tensor into multiple lower dimension tensors, and then fills in the missing value on the lower dimension tensors. Liu J, Musialski P, Wonka P, et al first come up with High Accuracy Low Rank Tensor Completion (HaLRTC), which extends the matrix case to the tensor case by proposing the first definition of the trace norm for tensors and then by building a working algorithm [8]. Though the accuracy of this kind of approach is high, the computation complexity is sometimes unbearable.

2.2. Deep learning method

In recent years, Deep Learning methods have made noticeable achievement from all aspects, and various DL approaches have been improved to deal with data imputation issue. [9] utilizes BRITS, an approach based on recurrent neural networks for missing values in time series data without any specific assumption. Though the bidirectional temporal dependency within time series is considered during data imputation, spatial dependency among different time series is ignored. [10] first transforms the raw data into spatial–temporal images and then implements data imputation task with the help of convolutional neural network (CNN). The weakness of this method is that short-term temporal features can be learned, while spatial features are limited to euclidean space. In [11], a deep auto-encoder for missing spatio-temporal data imputation is put forward, and the auto-encoder is designed with a combination of CNN and Bi-LSTM. Though the spatial and temporal dependencies can be extracted at the same time, it still cannot deal with non-euclidean spatio-temporal data, due to the employment of CNN. Moreover, [26] deploys CNN based GAN to reconstruct missing multivariate time series. Though GAN shows its advantage of constructing data distribution, only limited spatial relations can be learned due to 2D-convolution and the temporal receptive field is relatively short compared with RNN based models. Intelligent Transportation System (ITS), as a typical IoT application scene, suffers from data missing a lot [39], therefore, intensive Deep-Learning-based works have been conducted to resolve data imputation issue in this field. [42] uses a bidirectional Long Short-Term Memory network and Graph Neural Network to efficiently learn the spatial–temporal correlations in partially observed network traffic data. [35] imputes GPS trajectory data from probe vehicles by combining VAE with a new learning strategy, called self-interested coalitional learning (SCL). SCL forges cooperation between a main self-interested semi-supervised learning task and a discriminator as a critic to facilitate main task training while providing interpretability on the results. [40] designs a Multi-Attention Tensor Completion Network (MATCN) for modeling multi-dimensional representation of road sensory data that in the presence of missing entries. MATCN sparsely sampled historical fragments and utilized a gated diffusion convolution layer to generate the initial schemes, which mitigate the exposure bias existing in other traffic imputation models. It is costly to develop traffic data imputation models with high accuracy, since traffic data must be large and diverse, which is costly. Therefore, [41] use parallel data with generative adversarial networks (GANs) to enhance traffic data imputation. Parallel data is consist of synthetic data and real data, and the former is cheap, easy-access and generated by GANs based on the latter. With the help of synthetic data to augment the original data, the model can impute traffic data with a higher accuracy.

3. Preliminaries

3.1. Problem statement

As mentioned above, the spatio-temporal data are collected by various sensors embedded in different devices. For each sensor, data are collected and stored as time series. In this paper, we utilize two kinds of data: **Monitor Data** and **External Environment**. Monitor Data are typically spatio-temporal data collected by sensors in device network, defined as $\boldsymbol{X^{mon}} = \{X_1^{mon}, X_2^{mon}, X_3^{mon}, \ldots, X_T^{mon}\}$, where T is the length of each time series. The dimension of each $X_t^{mon} = \{x_t^{mon\text{-}1}, x_t^{mon\text{-}2}, x_t^{mon\text{-}3}, \ldots, x_t^{mon\text{-}M}\}$ is M^*N and of each $x_t^{mon\text{-}m}$ is N, denoting that there are M devices totally, and each device contains N various sensors. External Environment is shared by all devices in the same group, thus they are multivariant time series data, defined as $\boldsymbol{X^{ex}} = \{X_1^{ex}, X_2^{ex}, X_3^{ex}, \ldots, X_T^{ex}\}$, where T is the length of each time series. The dimension of each X_t^{ex} is K, denoting that External Environment consists of K different indexes. The objective of this paper is to populate the missing value in $\boldsymbol{X^{mon}}$ according to existing Monitor Data and External Environment.

3.2. Missing data type

According to [12], three missing data categories can be summarized: Missing Completely at Random (MCAR), Missing at Random (MAR) and Missing Not at Random (MNAR). If the missing does not depend on observed or unobserved data, that is MCAR, just like flipping a coin to guess which side is up. MAR refers to the missing that can be described using observed data. For example, the missing of spouse age data may depends on marital status data. When the missing depends on unobserved attribute or on the missing attribute itself [14], we call it MNAR. MNAR occurs when devices are in long-term failure. Our task mainly aims at the former two kinds of missing, since it is of little significance to populate the missing value in MNAR.

3.3. Variational Auto-Encoder

Variational Auto-Encoder (VAE) is a deep Bayesian generative model which has been successfully applied to many generally auxiliary tasks. The traditional VAE can be viewed as two coupled but independently parameterized models: the encoder (or recognition model), and the decoder (or generative model) [13]. These two models support each other. The VAE sends a high-dimensional input x into the encoder to get a latent representation z of low-dimension, and then uses the decoder to reconstruct x based on z.

In detail, the encoder of VAE intends to learn the posterior distribution $p(x|z)$, since the reconstructed x is sampled from it. However, it is intractable to compute $p(x|z)$, so VAE approximates it to $q(z|x)$. The loss function of standard VAE is defined in *Eq. (1)* as follows, and $p(z)$ is usually assumed to be Gaussian distribution. D_{KL} stands for KL divergence, which is used to measure the similarity between two distributions.

$$\begin{aligned} L(x) &= \mathbb{E}_{q(z|x)}(log(p(x|z))) - D_{KL}(q(z|x)||p(z)) \\ &= \mathbb{E}_{q(z|x)}(log(p(x|z))) + log(p(z)) - log(q(z|x)) \end{aligned} \tag{1}$$

We can use Monte Carlo integration to simplify the expectation above, as shown in *Eq.2*, in which z^i (i = 1,2...n) is sampled from the $q(z|x)$.

$$L(x) \approx \frac{1}{n}\sum_{i=1}^{n}(log(p(x|z^i))) + log(p(z^i)) - log(q(z^i|x)) \tag{2}$$

4. Methodology

In this section, the whole architecture of our ST-VAE is introduced in detail, which contains two parts: AM-VAE based data pre-processing and DI-VAE based data imputation. *Section 4.1* shows the overall workflow. *Section 4.2* presents the utilization of AM-VAE in data pre-processing phrase. *Section 4.3 and 4.4* describe the DI-VAE and Normalization Flow that used in data imputation phrase respectively.

4.1. Overall workflow

The whole workflow of ST-VAE is depicted in Fig. 1, and can be mainly divided into two parts: **Data Pre-processing** and **Data Imputation**.

During the Data Pre-processing, AM-VAE is utilized to process Monitor Data $\boldsymbol{X^{mon}} = \{X_1^{mon}, X_2^{mon}, X_3^{mon}, \ldots, X_T^{mon}\}$ into adjacent matrix A^{mon}. Besides, we also need to reshape Monitor Data into feature matrix F^{mon}. Both are the essential inputs for GCN in the following Data Imputation part.

Then the adjacent matrix A^{mon} and the feature matrix F^{mon} are fed into Data Imputation part, together with External Environment $\boldsymbol{X^{ex}} = \{X_1^{ex}, X_2^{ex}, X_3^{ex}, \ldots, X_T^{ex}\}$ to fill in the missing value in Monitor Data. The workflow of Data Imputation may also be subdivided into three stages: Encoder, Feature Transformation and Decoder. The Encoder and Decoder are complemented with DI-VAE (elaborated in *Section 4.3*), and Normalization Flow (elaborated in *Section 4.4*) is deployed in Feature Transformation.

4.2. AM-VAe

AM-VAE is utilized to pre-process the Monitor Data into adjacent matrix, which is required as the origin input when deploying GCN (further discussed in *Section 4.3*) to capture non-euclidean spatial dependencies of Monitor Data in DI-VAE.

Most existing works assume the graph structure is well established and given, yet this condition could often be violated in real-world scenarios [38]. In [15] adjacent matrix is the priori knowledge in Social Network Scene and in [17] adjacent matrix is presented as roads between different traffic sections, but these are non-universal with IoT. [16] calculates the adjacent matrix for Offshore Wind Power scene by setting a threshold to distance between turbines and [18] defines the adjacent matrix by measuring the consuming time between two regions in traffic, both of which might be too simple and subjective for device network in IoT. In order to apply GCN to predict citywide passenger demand, [19] utilizes Pearson Coefficient to measure the similarity of two time series, and set a threshold to obtain adjacent matrix. However, only limited time stamps are included in Pearson Coefficient, making it difficult to reflect the real spatial relations within device network.

Unlike these scenes applying GCN above, we deploy AM-VAE and JS divergence to obtain the adjacent matrix corresponding to the non-euclidean spatial relations of device network. The essence of our method is to utilize collected data similarity to measure the spatial relation of devices. Firstly, we feed M multivariate time series $x^{mon\text{-}1}, x^{mon\text{-}2}, \ldots, x^{mon\text{-}M}$ from M different devices into M different AM-VAEs to get different distributions of hidden variable z, which can represent the distribution of each $x^{mon\text{-}m} = \{x_1^{mon\text{-}m}, x_2^{mon\text{-}m}, x_3^{mon\text{-}m}, \ldots, x_T^{mon\text{-}m}\}$ corresponding to different devices when the model is well-trained. Then the adjacent matrix of device network can be defined by calculating the JS divergence between distributions of hidden variable z:

Different from DI-VAE, we only utilize GRU to complement the AM-VAE network, since time series from different devices are sent to different AM-VAEs, thus only the temporal feature is needed. What's more, all the timestamps value within one device share the same mean μ and variance σ^2, because we hope to figure out the overall data distribution at the device level. Fig. 2 shows the major architecture difference between DI-VAE and AM-VAE.

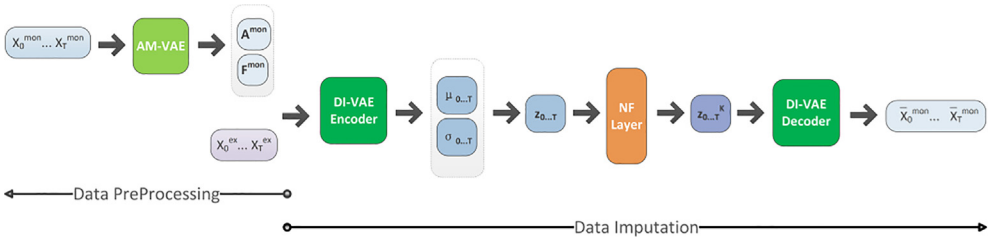


Fig. 1. Overall architecture of ST-VAE, the left part is data preprocessing and the right part is data imputation.

Supposing that we have two multivariate time series X_1 and X_2 from two different devices, P_1 and P_2 are feature distributions learned by AM-VAEs respectively, and $P_1 \sim (\mu_1, \sigma_1^2)$, $P_2 \sim (\mu_2, \sigma_2^2)$. We define $P_3 = (P_1 + P_2)/2$ and $P_3 \sim ((\mu_1 + \mu_2)/2, (\sigma_1^2 + \sigma_2^2)/2)$. Then we can calculate the JS divergence of P_1 and P_2 by *Eq.3* below, in which D_{KL} means KL divergence, and can be defined as $D_{KL}(P//Q) = -\sum_{x \epsilon X} P(x) log 1/Q(x) + \sum_{x \epsilon X} Q(x) log 1/P(x)$.

$$JS(z_1, z_2) = \frac{1}{2} KL\left(P_1 || \frac{P_1 + P_2}{2}\right) + \frac{1}{2} KL\left(P_1 || \frac{P_1 + P_2}{2}\right)$$
$$= \frac{1}{2} D_{KL}(P_1 || P_3) + \frac{1}{2} D_{KL}(P_1 || P_3) \tag{3}$$

$$= -\frac{1}{4}\left(log \frac{\sigma_1^2}{\sigma_3^2} - \frac{\sigma_1^2}{\sigma_3^2} - \frac{(\mu_1 - \mu_3)^2}{\sigma_3^2} + 1\right)$$
$$-\frac{1}{4}\left(log \frac{\sigma_2^2}{\sigma_3^2} - \frac{\sigma_2^2}{\sigma_3^2} - \frac{(\mu_2 - \mu_3)^2}{\sigma_3^2} + 1\right)$$

$$= -\frac{1}{4}\left(log \frac{16\sigma_1^2\sigma_2^2}{(\sigma_1^2 + \sigma_2^2)^2} - \frac{4(\sigma_1^2 + \sigma_2^2) + 2(\mu_1 - \mu_2)^2}{\sigma_1^2 + \sigma_2^2} + 2\right)$$
$$= -\frac{1}{2}\left(log \frac{4\sigma_1^2\sigma_2^2}{(\sigma_1^2 + \sigma_2^2)^2} - \frac{(\mu_1 - \mu_2)^2}{\sigma_1^2 + \sigma_2^2} - 1\right)$$

Then the similarity between two time series from different devices can be measured by *JS* (z_1, z_2) and adjacent matrix A^{mon} can be calculated correspondingly by *Eq.* (4):

$$A_{ij}^{mon} = \begin{cases} 1, if\ \ JS(z_i, z_j) > \beta \\ 0, otherwise \end{cases} \tag{4}$$

In order to reshape the Monitor Data into feature matrix, the data collected from various sensors within the same device are list in the same row, with different device data in different rows, as is shown in Fig. 3.

4.3. Di-VAE

As mentioned in *Section 3.3*, the basic framework of VAE consists of two parts: Encoder and Decoder. In DI-VAE, we utilize GCN and Multi-head GRU to implement the Encoder part, while Multi-head GRU for the Decoder part. GCN is used to extract non-euclidean spatial features and Multi-head GRU is for temporal features, thus overall features are obtained sufficiently from spatio-temporal data in IoT. It should also be noticed that, different from traditional auto-encoder, DI-VAE employs more generative generation model, which adds much stochasticity to the filled missing value and make them more realistic.

Fig. 4 shows the details of Encoder in DI-VAE. Adjacent matrix $\boldsymbol{A^{mon}}$ and feature matrix $\boldsymbol{F^{mon}}$ are sent to multiple layers of GCN to catch the non-euclidean spatial feature of Monitor Data, and then fed to Multi-head GRU to learn the temporal dependency device by device. In this way, the spatio-temporal feature $h_0^{mon} \ldots h_T^{mon}$ of Monitor Data is achieved. Different from Monitor Data, we only extract temporal features $h_0^{ex} \ldots h_T^{ex}$ of External Environment with the help of GRU, for there is little spatial dependency among them. Finally, we fuse them together in *Eq.* (5). Due to the different dimensions between both hidden states, and in order to maintain the relatively independent effects of them in the result, we concatenate both two together as follows. α and β are weight parameters that needs to be trained together with the DI-VAE.

$$\boldsymbol{h^{comb}} = \begin{pmatrix} \alpha * \boldsymbol{h^{mon}} \\ \beta * \boldsymbol{h^{ex}} \end{pmatrix} \tag{5}$$

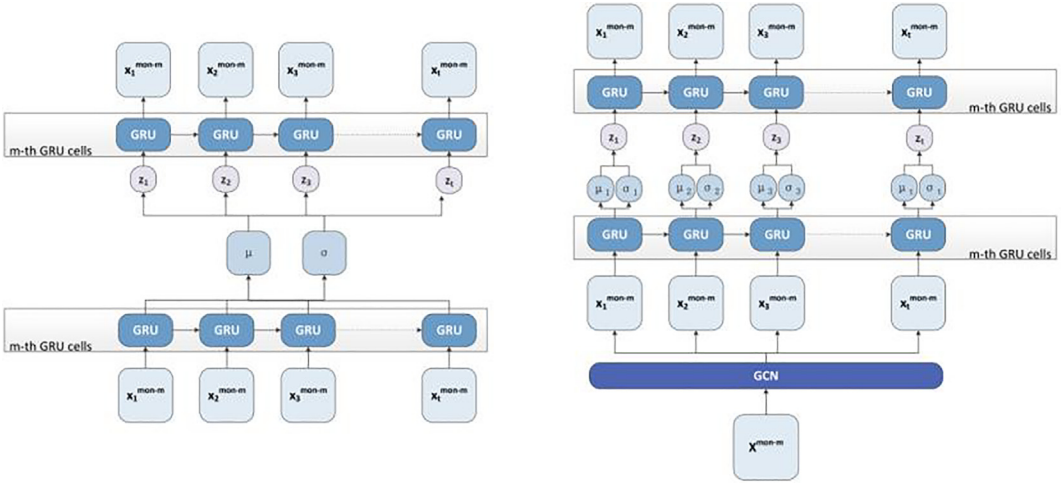


Fig. 2. Comparison between AM-VAE and DI-VAE. The left part is AM-VAE and the right part is DI-VAE.

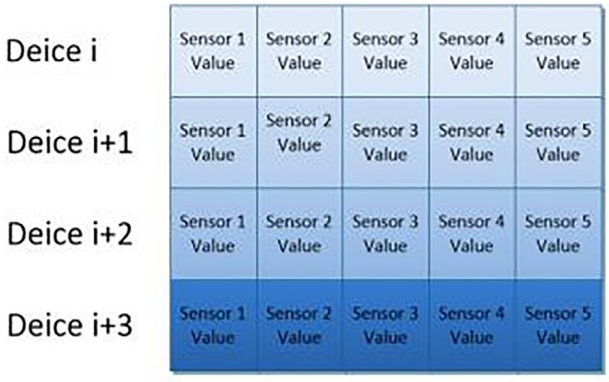


Fig. 3. Feature Matrix format of Monitor Data.

Different from traditional encoder in VAE, not only the hidden states $h_0^{comb}...h_T^{comb}$ of Multi-head GRU are time ordered, we also add temporal dependency into latent variables $z_0...z_T$. According to standard VAE, each z_t is sampled from Gaussian Distribution respectively, thus there is no temporal dependencies among them. To resolve the problem above, we concat h_t^{comb} and z_{t-1} before sending to the Full Connection Layer (FCN) to learn the distribution (μ_t and σ_t) of z_t just as shown in *Eq. (6)*. By contacting h_t^{comb} and z_{t-1}, sampled z_t can be temporal related with previous z_0 to z_{t-1}.

$$\mu_t = w^{\mu}\left[z_{t-1}, h_t^{comb}\right] + b^{\mu}, \sigma_t = ReLU\left(w^{\sigma}\left[z_{t-1}, h_t^{comb}\right] + b^{\sigma}\right) \tag{6}$$

For the Decoder in DI-VAE, only the Multi-head GRU is used to decode latent variable $z_0...z_T$ back into reconstructed Monitor Data $\boldsymbol{X^{mon}}$. As a result, all the missing value in original Monitor Data have been filled in the reconstructed Monitor Data. In the following part, we will introduce how two major techniques are employed in DI-VAE: Graph Convolution Network (GCN) and Multi-head Gated Recurrent Unit (GRU).

GCN. Firstly, adjacent matrix and feature matrix deriving from Monitor Data are fed into GCN [27]. In traditional GCN network, we calculate each hidden layer l by *Eq. (7)*, in which A refers to adjacent matrix and W refers to parameter matrix that needs to be trained. Besides, H_0 is set to be feature matrix $\boldsymbol{F^{mon}}$.

$$H^l = \sigma\left(AH^{l-1}W^{l-1}\right) \tag{7}$$

However, there remains two major problems when using *Eq. (8)*. On the one hand, it cannot transfer a node's own features to the next layer through multiplying an adjacency matrix A with a hidden-feature matrix H. On the other hand, values of feature map might grow rapidly through layer-by-layer extension. Therefore, [20] gives a solution to apply adjacent matrix $A' = I + A$ to replace A and add an additional spectral normalization scheme, in which I is an identity matrix. The optimized equation is shown as follows.

$$H^l = \sigma\left(D^{-\frac{1}{2}}A\prime D^{-\frac{1}{2}}H^{l-1}W^{l-1}\right), D_{ii} = \sum_j A\prime_{ij} \tag{8}$$

Multi-head GRU. Gated Recurrent Unit (GRU), as a subcategory of RNN, not only shows the ability to memorize the past events like RNN, but also covers the shortcoming of RNN when dealing with long-term dependency problems. Moreover, contrasted with LSTM, another kind of RNN, GRU performs equally well (sometimes even better in some typical applications), but has fewer parameters and simpler structure, therefore is more resource-efficient to train the model. Different from language sentences or video sequence, data collected by sensor is seldom of high-dimension or complex-feature, thus simple structure of GRU can easily handle. Therefore, GRU is chosen in this work to capture the temporal dependency of Monitor Data and External Environment in IoT. Unlike LSTM, GRU has only two kinds of gates: reset gate r_t and update gate z_t. We use the follow equations to calculate the hidden state $\boldsymbol{h^{mon}}$ and $\boldsymbol{h^{ex}}$ of Monitor Data and External Environment.

$$r_t = \sigma(W_r \cdot [h_t, x_t]), z_t = \sigma(W_z \cdot [h_t, x_t]) \tag{9}$$

$$h_t' = tanh(W_h \cdot [r_t * h_{t-1}, x_t]) \tag{10}$$

$$h_t = (1 - z) \odot h_{t-1} + z \odot h_t' \tag{11}$$

Feature maps output by GCN are divided into different groups according to different devices, and then sent to Multi-head GRU to capture temporal features, with one-head corresponding to one-device. Multi-head GRU employed in this paper mainly works for two reasons: On the one hand, data collected from different devices might vary a lot in nature, so it is unreasonable to feed them into one set of GRUs and share the same parameter matrix. On the other hand, this Multi-head GRU structure adds more transfer flexibility to our network, since configuration of devices in IoT often varies when devices are installed removed or modified [20].

What's more, it is worth noticing that we use $h_0^{comb}...h_T^{comb}$ with *0-T* timestamps as the hidden states in auto-encoder, instead of the last timestamp h_T^{comb}, just as [19] do. The reason is that according to [21], when applying RNN as auto-encoder, hidden states of multiple timestamps perform much better than just the last timestamp hidden state.

4.4. Normalization flow

As discussed in *Section 3*, hidden variable z is sampled from $q(z|x)$, which is often assumed to be Gaussian distribution in traditional VAE. However, this assumption makes the network less robust because the $q(z|x)$ may not always follow Gaussian distribu-

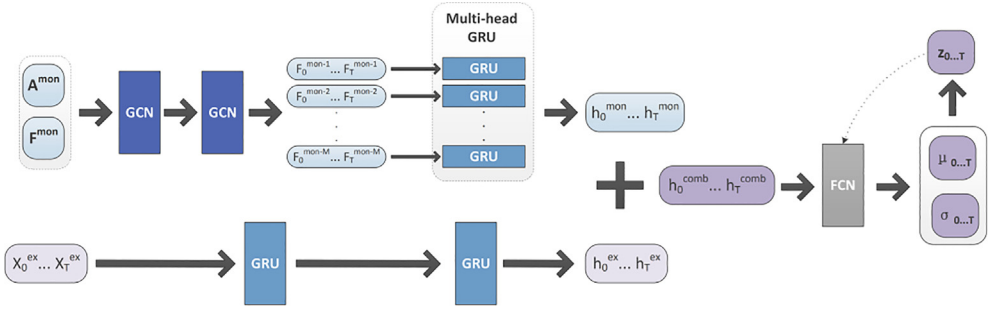


Fig. 4. Detail of Encoder in DI-VAE, the upper part is Monitor Data workflow and the lower part is External Environment workflow, and hidden states output by both workflows are combined to calculate combined hidden states $\boldsymbol{h^{comb}}$. Finally, "current" combined hidden state h_t^{comb} and "past" latent variable z_{t-1} are sent to FCN to learn the μ_t and σ_t according to *Eq. (6)*.

tion. In order to solve this problem, we utilize normalization flow (NF) to transform $q(z|x)$ into regular distribution through several invertible mappings. NF applies a series of transformations $\{g^1, g^2 \ldots, g^K\}$ to construct arbitrarily complex distribution $z^K = g^K \circ g^{K-1} \circ \cdots \circ g^0(z^0)$, and we can get the following equation:

$$logp(z^K) = logp(z^{K-1}) - log\left|det\frac{\partial g^K}{\partial z^{K-1}}\right| = logp(z^0) - \sum_{k=1}^{K} log\left|det\frac{\partial g^k}{\partial z^{k-1}}\right| \quad (12)$$

In particular, we select Linear NF for the reason that it can express any form of correlation between dimensions and data in IoT, and it is simple in structure. Linear NF is defined as follows:

$$g(z) = Az + b \quad (13)$$

where $A \in \mathbb{R}^{D*D}$ and $b \epsilon \mathbb{R}^D$ are parameters that need to be trained with the network, and A is an invertible matrix. However, it should be noticed that the determinant of the Jacobian is simply $det(A)$, which can be computed in $o(D^3)$, as can the inverse [22]. It is widely acknowledged that convolutions are easy to compute inversely and the determinant are unobvious, so Zheng et al. [23] explore 1D convolutions to compute the determinant of the Jacobian effectively. We optimize linear NF following their method as in *Eq.14*, in which $w \epsilon \mathbb{R}^k$ is the filter of the 1-D convolution, h is the activation function and $u \epsilon \mathbb{R}^d$ is a vector adjusting the dimension of output from the activation function.

$$g(z) = z + u \odot h(conv(z, w)) \quad (14)$$

5. Experiments

In this section, the experiment datasets we based on and related pre-processings are conducted in detail. The experiments are mainly divided into two types: ablation experiments and comparison experiments. The former is used to verify the effectiveness of some essential methods that used in our network, and the latter is to illustrate that ST-VAE outperforms other state-of-art approaches.

5.1. Datasets

Real-World Datasets. As for ablation experiment, we deploy real-world datasets in geological disaster monitoring scene of IoT, in which data collected by sensors show both temporal and spatial features. We collect Monitor Data and External Environment from January to December in *2019*, and the acquisition interval is *1 h*. In terms of Monitor Data, we trace *36* devices in total, with *16* devices in Men Tou Gou District of Beijing City, *10* devices in Shi Yan City of Hubei Province and *10* devices in Yi Chang City of Hubei Province. Devices installed in the same city consist of the same type of sensors, whereas differ other cities. Types of sensors embedded in devices for three cities are listed in Table 1. In terms of External Environment, we concentrate on the temperature, moisture, illumination intensity and wind strength, which show potential dependencies on Monitor Data collected by sensors.

Public Datasets. The contrast methods that we choose do not take External Environment into consideration. To be fair, we select two different public datasets for comparison experiments. GHS [25] only consists of Monitor Data, while PRSA [24] consists of both Monitor Data and External Environment. Moreover, both two datasets are originated from UCI[1].

PRSA [24] dataset includes hourly air pollutants data from *12* nationally-controlled air-quality monitoring sites located in Beijing China. The time range is from March *1st 2013* to February *28th 2017*. Six kinds of sensor-collected data (PM2.5, PM10, SO_2, NO_2, CO, O_3) are regarded as Monitor Data, while five other kinds of meteorological data (Temp, Pressure, Degree, Rain, Wind direction, Wind speed) collected from the nearest weather stations are selected as External Environment that used in ST-VAE.

Table 1
Sensor Type of Different Devices in Real-world Dataset.

City(District)	*Sensor Type*	*Description*
Men Tou Gou	GNSS Displacement	Monitoring surface displacement from X, Y and H direction
	Clinometer	Monitoring underground displacement from X and Y direction
Shi Yan	Rain Gauge	Monitoring rain capacity
	Inclinator	Monitoring the angular change of the ground from X, Y and Z direction.
Yi Chang	Soil Moisture	Monitoring soil moisture from different depth (1 m, 1.5 m, 2 m, 2.5 m, 3 m)
	Displacement Meter	Monitoring surface displacement

According to the Table above, data dimension collected by devices located in three cities is *5(3 + 2)*, *4(1 + 3)* and *6(5 + 1)*.

GHG [25] dataset contains time series of greenhouse gas concentrations at *2921* grid cells in California during the period of May *10th* to July *31th* in *2010*. Each grid cell covers an area of *12* km by *12* km, and one data file corresponds to one grid cell. In terms of data files, we randomly select seven files (*82, 93, 312, 645, 1573, 2099, 2583*), and value at each timestamp consists of *16* indexes. All the *16* indexes are considered as Monitor Data for ST-VAE.

Data Pre-processing. Neither the real-world datasets nor public datasets contain any missing value originally. We use half of them as training data, and the rest are for testing. In the testing part, we drop some value at random before the ablation and comparison experiments and the corresponding positions are marked. The missing rate is set to *5 %* in real-world datasets by default and set to *5 %–50 %* in public datasets, because the former is used to verify the effectiveness of major techniques in ST-VAE, while the latter is used to compare the performances of our method with other classical methods under different missing rates. Moreover, in real IoT practice, if more than half of the data collected by sensor are missing, the sensor is regarded as fault and ought to be replaced. So, it is meaningless to populate missing value when missing rate is higher than *50 %*. What's more, all the experiments are under the assumption that External Environment is objective without missing value.

5.2. Experiment set up

All the experiments are coded with Python 3.8.5, and we choose PyTorch as the deep learning framework to implement our ST-VAE (including VAE, GCN and GRU) and other comparison approaches.

1 https://archive.ics.uci.edu/ml/datasets.php.

In AM-VAE part, hidden state $h_{0\text{-}t}$ of all the timestamps within one Multi-head GRU correspond to the same mean μ and variance σ^2, as shown in Fig. 2. The dimension of latent variable z is set to be smaller than the output h of GRU, and hyper parametric search is utilized to get the best performance according to different datasets. For real-world dataset, z and h is set to be *4* and *8* respectively. For public dataset, z and h is set to be *4*, *8* for PRSA and *8, 16* for GHG.

In DI-VAE part, hidden state $h_{0\text{-}t}$ of all the timestamps within one Multi-head GRU are sent to the same FCN but receive different mean μ_t and variance σ_t^2 respectively as shown in Fig. 2. As for Multi-head GRU, the number of GRUs is the same as the number of devices. For each GRU, dimension of hidden state h^m output by

Monitor Data GRU is set to *16*, while dimension of hidden state h^e output by External Environment GRU is set to *4*. As for GCN, only the shape of weight matrix W is hyper-parameter, and n^*n (n is the number of sensors within one device) is set so that the output of each GCN layer conform to the same shape according to *Eq.4*. Besides, lantent variable z is set to *8* and the length of NFs is *4*.

Considering the feature of data imputation task, we choose Root Mean Squared Error (RMSE), Normalization Mean Squared Error (NMSE) and Mean Absolute Error (MAE) to evaluate the performance of ST-VAE and other approaches, which can be calculated by *Eqs. (15)–(17)*. RMSE and MAE are used to calculate the relative error of multiple time stamps value collected by all the sensors that within single device as a whole, while NMSE is used to calculate the absolute error of single time stamp value collected by single sensor. However, compared with MAE, RMSE is relatively more sensitive to 'abnormal' data, so high RMSE means extremely 'bad' padding data.

$$RMSE = \sqrt{\frac{1}{m}\sum_{i=1}^{m}\left(x_i - \widehat{x_i}\right)^2} \tag{15}$$

$$NMSE = \frac{\sum_{j=1}^{J}\sum_{i=1}^{I}\left(x_{i,j} - \widehat{x_{i,j}}\right)^2}{\sum_{j=1}^{j}\sum_{i=1}^{i}\left(x_{i,j}\right)^2} \tag{16}$$

$$MAE = \frac{1}{m}\sum_{i=1}^{m}\left|\left(x_i - \widehat{x_i}\right)\right| \tag{17}$$

Besides, we test all the assembly of hyper-parameter and threshold to ensure the best performance, and the same operation is performed when dealing with other approaches. All the hyper-parameters involved in some baseline methods of comparison experiments is shown in Table 2, and the hyper-parameters of the rest deep learning baseline methods are according to the configurations in the original papers.

6. Evaluations

6.1. Ablation experiments

In order to verify the effectiveness of some major techniques existing in our network, we reconfigure ST-VAE to create five variants to conduct ablation experiments described as follows: 1) GCN; 2) AM-VAE; 3) External Environment; 4) z-concatenation; 5) Normalization Flow. The results of ablation experiments are as follows.

GCN. As described in *Section 4*, we deploy GCN to capture non-euclidean spatial dependency among devices. In ablation experiment, we remove the GCN block from DI-VAE. Multi-variate time series of Monitor Data are directly fed to the Multi-head GRU separately device by device. From the result in Table 3, it is obvious that its performance on RMSE indexes is barely satisfactory, and is even worse on MAE and NMSE indexes, demonstrating the importance of catching non-euclidean dependency among time series from different devices in IoT.

Table 2
Hyper-parameters of Comparison experiments.

Baseline Method	Parameter	Value	
		GHG	*PRSA*
ARIMA	start_q, max_p, max_q	*2, 5, 5*	
KNN	k	*4*	
PPCA	No. of principle componet	*2*	*6*
HaLRTC	rho	*1e-4*	

Table 3
Results of Ablation Experiment.

Target	*RMSE*	*MAE*	*NMSE*
GCN	47.6550	10.2726	0.4263
AM-VAE	73.4412	5.0167	0.2649
External Environment	34.6697	6.6641	0.1701
z-concatenation	20.5206	4.3321	0.0346
Normalization Flow	29.3417	4.8824	0.0211
ST-VAE	**17.2151**	**3.4846**	**0.0124**

AM-VAE. According to *Section 4*, adjacent matrix is the essential input of GCN, while there is little prior knowledge about it in IoT. Nevertheless, we apply AM-VAE to deal with this puzzle with the method mentioned in *Section 4.2*. In order to verify its absolute advantage over other similarity measurement approaches, we replace it with Pearson Coefficient, which is used in [19], to measure the similarity among different time series. Data in Table3 suggest that its performance on RMSE remains the worst. The inaccuracy of calculating some certain device spatial relation (compared to AM-VAE) results in the extremely inaccurate imputation of some value, which further leads to the unacceptable performance of RMSE (elaborated in *Section 5.2*).

Normalization Flow and z-Concatenation. Based on the theory of basic VAE in *Section 3*, latent variable z is sampled from Gaussian distribution, which is over-specialization. On the one hand, in order to add more robustness to ST-VAE, we apply normalization flow to map Gaussian distribution into more general distribution (elaborated in *Section 4.4*). On the other hand, for the sake of adding temporal dependency into z, we concatenate h_t with z_{t-1} when generating z_t (elaborated in *Section 4.3*). To illustrate the effective optimizing of z, we remove the relative approaches from DI-VAE. It can be seen from the Table 3 that ST-VAE outperforms both ablation experiments in all targets, demonstrating the necessity of both techniques.

In general, ablation experiment results indicate that all the five major techniques mentioned above play important roles in the ST-VAE, making it an effective approach to complete the spatio-temporal data imputation task in IoT.

6.2. Comparison experiments

ARIMA and KNN. ARIMA predicts missing value based on a period of past values from the same device and KNN fills in the missing value with the most 'similar' one that within the same device network. Both approaches take only temporal or spatial dependency into consideration. The performances of these two approaches are quite similar on two public datasets, but far from that of ST-VAE in RMSE, MAE and NMSE, suggesting the importance of spatial–temporal features in IoT data.

PPCA. PPCA [5] tends to complete the data imputation task by building the distribution of time series. However, due to its limited statistic capacity, the performance is worse compared with no matter traditional methods or deep learning methods in general.

HaLRTC. HaLRTC [8] turns out to be the most effective approach among all the traditional approaches overall. However, comparing with ST-VAE, the performance of HaLRTC is not stable when dataset varies (good in GHG while bad in PRSA). What's more, as the missing rate increases, the results rapidly get worse (such as RMSE is from *41.3380* to *339.8110*), while ST-VAE is relatively stable according to Fig. 5 and Fig. 6.

Convolution Bidirectional-LSTM. CB-LSTM in [11] utilizes CNN to extract spatial features and Bidirectional-LSTM to extract temporal features. However, CNN is limited to euclidean space and external environment influence is not considered, both of which are crucial in IoT scene. That is why ST-VAE outperforms CB-

LSTM only a little in GHG dataset but much more in PRSA dataset, since External Environment of PRSA is combined as input for ST-VAE. What's more, CB-LSTM is based on discriminative model, which is less capable of generalization than ST-VAE (Fig. 7).

CNN-GAN. [26] employs CNN to implement GAN based generative network but the shortcoming is obvious. On the one hand, CNN can only catch limited length of temporal dependency compared with GRU in ST-VAE, and non-euclidean spatial dependency cannot be obtained. On the other hand, GAN is relatively hard to convergence in training phrase compared with VAE, which is also a type of generative model. The result from Fig. 5 and Fig. 6 also confirm that ST-VAE significantly outperforms CNN-GAN.

MLP-VAE. [28] is a deep Bayesian model that could learn stochastic mappings between observed data and latent variables. Due to the stochastic representations of VAE, the overall performance of MLP-VAE is second only to ST-VAE, demonstrating the advantage of VAE with generalization ability in data imputation tasks. However, it is still not as good as ST-VAE for two reasons: On the one hand, hidden variable in MLP-VAE obeys the Gaussian distribution, making the model less generalized, when compared with Normalization Flows used in ST-VAE. On the other hand, multi-layer perceptions are less satisfactory than GRU and GCN when extracting spatio-temporal features.

DeepSTN+. [36] is a auto-encoder-based model for traffic data prediction. Multiple layers of spatial convolutions are employed in DeepSTN+ to learn global spatial features, and temporal features are also learned by temporal convolution at the same time. However, ability of convolution is limited when leaning temporal features and only non euclidean spatial features can be extracted, compared with ST-VAE. Moreover, since DeepSTN+ is a prediction based model, in which missing value are imputed by predicted ones. When missing rate is increasing, the performance of it turns out to be bad according to Fig. 5 and Fig. 6. In contrast, as a reconstruction-based model, ST-VAE performs steadily under different missing rates.

Moreover, it is obvious that all the approaches except for HALRTC perform steadily on PRSA dataset according to RMSE, MAE and NMSE results of Fig. 6, which is a unusual phenomenon. Two possible reasons can be assumed: On the one hand, dimension of Monitor Data in PRSA (6) is relatively small than that in GHG (16), therefore all the approaches can be fully qualified no matter what the missing rate is. On the other hand, External Environment

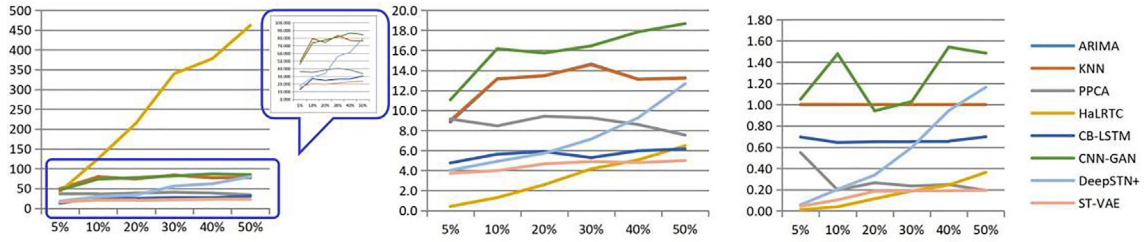


Fig. 5. Comparison Experiment Result of RMSE, MAE and NMSE on GHG Dataset with Missing Rate of 5%–50%.

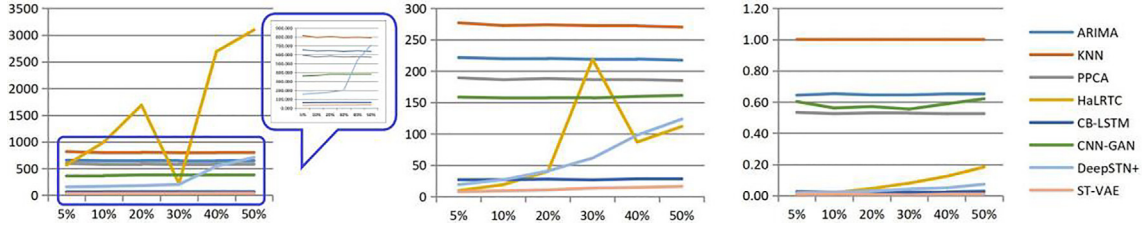
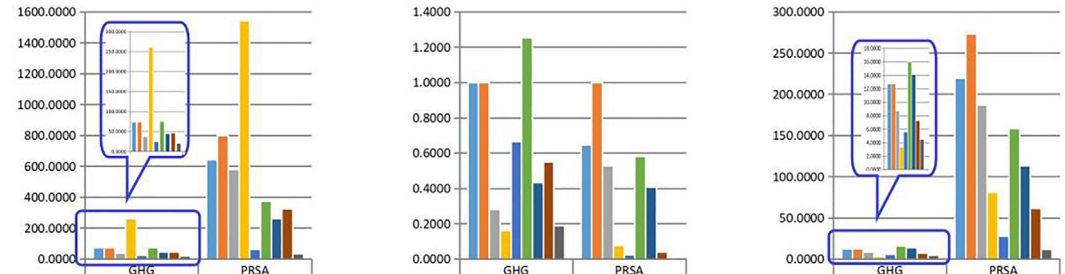


Fig. 6. Comparison Experiment Result of RMSE, MAE and NMSE on PRSA Dataset with Missing Rate of 5%–50%.



	GHG	PRSA
ARIMA	73.3090	644.2800
KNN	73.3570	800.4640
PPCA	37.0940	581.5740
HaLRTC	260.1810	1542.2340
CB-LSTM	24.7022	64.8671
CNN-GAN	75.0071	375.7708
MLP-VAE	44.7114	260.8147
DeepSTN+	46.3045	326.0329
ST-VAE	20.6258	34.4447

	GHG	PRSA
ARIMA	0.9993	0.6479
KNN	1.0008	1.0000
PPCA	0.2810	0.5280
HaLRTC	0.1600	0.0769
CB-LSTM	0.6667	0.0245
CNN-GAN	1.2539	0.5825
MLP-VAE	0.4343	0.4076
DeepSTN+	0.5508	0.0392
ST-VAE	0.1883	0.0088

	GHG	PRSA
ARIMA	12.7420	219.5100
KNN	12.7540	272.9610
PPCA	8.7360	186.9410
HaLRTC	3.3310	81.0840
CB-LSTM	5.6158	27.7051
CNN-GAN	15.9857	158.4958
MLP-VAE	14.1269	113.2494
DeepSTN+	7.2881	61.6902
ST-VAE	4.5102	12.0467

Fig. 7. Comparison Experiment Result of RMSE, MAE and NMSE on Two datasets with Average Missing Rate.

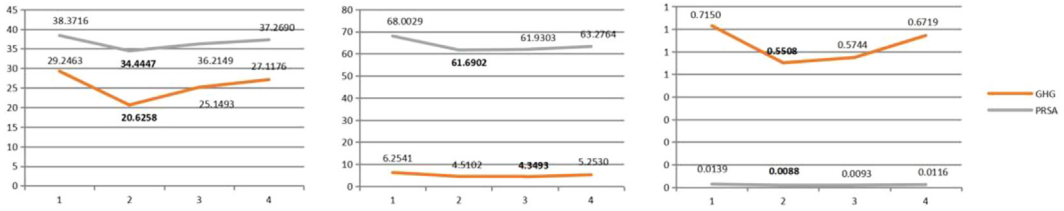


Fig. 8. Sensitive study of GCN layer n on two public datasets with Average Missing Rate. The measurement indexes are RMSE, MAE and NMSE from left to right.

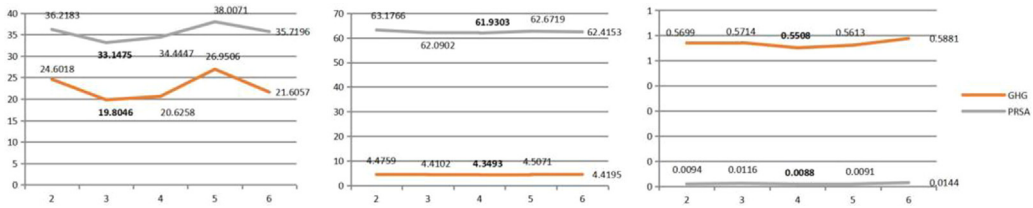


Fig. 9. Sensitive study of length of NFs l on two public datasets with Average Missing Rate. The measurement indexes are RMSE, MAE and NMSE from left to right.

is utilized as additional information in PRSA, which also proves its importance in data imputation of IoT.

From the discussion of comparison experiment, we can draw a conclusion that our ST-VAE outperforms other state-of-art approach generally, and we owe this to the framework that not only pay attention to both temporal and non-euclidean spatial features, but also take individual stochasticity into account, which fits IoT data perfectly.

6.3. Sensitive study

In order to evaluate the effectiveness of various parameterizations of ST-VAE, hyper-parameter sensitive study is conducted on public datasets with average missing rates. Two major hyper-parameter is selected as the object of the study: number of GCN layer n and length of NFs l, which determinate the basic architecture of ST-VAE. The results are as shown in Fig. 8 and Fig. 9. It can be observed from the results that too deep GCN and NFs may not improve the performance of the ST-VAE, therefore *2* is set for n and *4* is set for l.

7. Conclusion

Spatio-temporal data imputation is a laborious and profound research topic in IoT area. In this paper, we propose a novel ST-VAE, which consisting of AM-VAE and DI-VAE to deal with this problem. The former aims at calculating the adjacent matrix for spatial relations of device network which is the essential input of GCN, and the latter performs the data imputation task based on the non-euclidean spatial and temporal dependencies. What's more, linear normalization flow and temporal concatenation are optimized to latent variable z of VAE, making our network more robust and capable of stochastic generalization.

CRediT authorship contribution statement

Shuo Zhang: Conceptualization, Methodology, Validation, Formal analysis, Writing – original draft. **Jinyi Chen:** Software, Visualization. **Jiayuan Chen:** Software, Investigation. **Xiaofei Chen:** Data curation. **Hejiao Huang:** Supervision, Writing – review & editing, Project administration.

Data availability

The data that has been used is confidential.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work is financially supported by Shenzhen Science and Technology Program under Grant No. GXWD20220817124827001 and No. JCYJ20210324132406016.

References

[1] S. Zhang, X. Chen, J. Chen, Q. Jiang and H. Huang, Anomaly Detection of Periodic Multivariate Time Series under High Acquisition Frequency Scene in IoT, in International Conference on Data Mining Workshops, pp. 543–552, 2020.
[2] M. Zhong, S. Sharma, P. Lingras, Genetically designed models for accurate imputation of missing traffic counts, J. Transport. Res. Record 1879 (1) (2004) 71–79.
[3] S. Tak, S. Woo, H. Yeo, Data-driven imputation method for traffic data in sectional units of road links, IEEE Trans. Intell. Transp. Syst. 17 (6) (2016) 1762–1771.
[4] Y. Li, Z. Li, L. Li, Missing traffic data: comparison of imputation methods, IET Intell. Transport. Syst. 8 (1) (2014) 51–57.
[5] L. Qu, J. Hu, L. Li, Y. Zhang, PPCA-based missing data imputation for traffic flow volume: a systematical approach, IEEE Trans. Intell. Transp. Syst. 10 (3) (2009).
[6] L. Qu, Y. Zhang, J. Hu, A BPCA based missing value imputing method for traffic flow volume data, in IEEE Intelligent Vehicles Symposium. pp. 985–990, 2008.
[7] L. Li, Y. Li, Z. Li, Efficient missing data imputing for traffic flow by considering temporal and spatial dependence, Transport. Res. Part C: Emerg. Technol. 34 (2013) 108–120.
[8] J. Liu, P. Musialski, P. Wonka, Tensor completion for estimating missing values in visual data, IEEE Trans. Pattern Anal. Mach. Intell. 35 (1) (2013) 208–220.
[9] W. Cao, D. Wang, J. Li, H. Zhou, L. Li, Y. Li. BRITS: Bidirectional Recurrent Imputation for Time Series, in Advances in Neural Information Processing Systems, vol. 31, 2018.
[10] Y. Zhuang, R. Ke, Y. Wang, Innovative method for traffic data imputation based on convolutional neural network, IET Intelligent Transport. Syst. 13 (2018) 605–613.
[11] Asadi, Reza and A. Regan, A convolution Ecurrent auto-encoder for spatio-temporal missing data imputation, in ArXiv, abs/1904.12413, 2019.
[12] R.C. Pereira, M.S. Santos, P.P. Rodrigues, P.H. Abreu, Reviewing auto-encoders for missing data imputation, Techn. Trends, Appl. Outcomes 69 (2020) 1255–1285.
[13] D.P. Kingma, M. Welling, An introduction to variational auto-encoders, Found. Trends Machine Learn. 12 (4) (2019) 307–392.

[14] L. Gondara, K. Wang, MIDA: multiple imputation using denoising auto-encoders, Adv. Knowledge Disc. Data Min. (2018) 260–272, 10939.
[15] A. Chaudhary, H. Mittal, A. Arora, Anomaly Detection Using Graph Neural Networks, in International Conference on Machine Learning, Big Data, Cloud and Parallel Computing, pp. 346–350, 2019.
[16] M. Yu, Z. Zhang, X. Li, J. Yu, J. Gao, Z. Liu, B. You, X. Zheng, R. Yu, Superposition graph neural network for offshore wind power prediction, Futur. Gener. Comput. Syst. 113 (2020) 145–157.
[17] Z. Xie, W. Lv, S. Huang, Z. Lu, B. Du, R. Huang, Sequential graph neural network for urban road traffic speed prediction, IEEE Access 8 (2020) 63349–63358.
[18] T. Wu, F. Chen, Y. Wan, Graph Attention LSTM Network: A New Model for Traffic Flow Forecasting, in International Conference on Information Science and Control Engineering, pp. 241–245, 2018.
[19] L. Bai, L. Yao, S. Salil, X. Wang, W. Liu, Z. Yang. Spatio-Temporal Graph Convolutional and Recurrent Networks for Citywide Passenger Demand Prediction. ACM International Conference on Information and Knowledge Management, Association for Computing Machinery, pp. 2293–2296, 2019.
[20] M. Canizo, I. Triguero, A. Conde, E. Onieva, Multi-head CNN–RNN for multi-time series anomaly detection: an industrial case study, Neurocomputing 363 (2019) 246–260.
[21] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, D. Pei, Robust Anomaly Detection for Multivariate Time Series through Stochastic Recurrent Neural Network, in ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 2828–2837, 2019.
[22] I. Kobyzev, S. Prince and M. Brubaker, Normalizing Flows: An Introduction and Review of Current Methods, in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 43, pp., 2021.
[23] G. Zheng, Y. Yang, J. Carbonell, Convolutional Normalizing Flows, in CoRR, abs/1711.02255, 2017.
[24] X. Liang, T. Zou, B. Guo, S. Li, H. Zhang, S. Zhang, H. Huang and S. X. Chen, Assessing Beijing's PM2.5 Pollution: Severity, Weather Impact, APEC and Winter Heating, in: Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences, vol. 471, pp. 20150257, 2015.
[25] D.D. Lucas, C. Yver Kwok, P. Cameron-Smith, H. Graven, D. Bergmann, T.P. Guilderson, R. Weiss, R. Keeling, Designing optimal greenhouse gas observing networks that consider performance and cost, Geosci. Instrum. Methods Data Syst. 4 (1) (2015) 121–137.
[26] Z. Guo, Y. Wan, H. Ye, A data imputation method for multivariate time series based on generative adversarial network, Neurocomputing 360 (2019) 185–197.
[27] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, P.S. Yu, A comprehensive survey on graph neural networks, IEEE Trans. Neural Networks Learn. Syst. 32 (1) (2021) 4–24.
[28] G. Boquet, J. L. Vicario, A. Morell, J. Serrano. Missing Data in Traffic Estimation: A Variational Autoencoder Imputation Method, in IEEE International Conference on Acoustics, Speech and Signal Processing, pp. 2882–2886, 2019.
[29] H.M. Al-Deek, C. Venkata, S. Ravi Chandra, New algorithms for filtering and imputation of real-time and archived dual-loop detector data in I-4 data warehouse, Transport. Res. Record: J. Transport. Res. Board 1867 (2004) 116–126.
[30] L. Qu, L. Li, Y. Zhang, J. Hu, PPCA-based missing data imputation for traffic flow volume: a systematical approach, IEEE Trans. Intell. Transp. Syst. 10 (2009) 512–522.
[31] J. Xu, X. Li, H. Shi, Short-term traffic flow forecasting model under missing data, J. Computer Appl. 30 (2010) 1117–1120.
[32] J. Zhang, Y. Zheng, D. Qi, Deep Spatio-Temporal Residual Networks for Citywide Crowd Flows Prediction, in Proceedings of the 31th AAAI Conference on Artificial Intelligence, pp. 1655–1661, 2017.
[33] Y. Chen, Y. Lv, F.Y. Wang, Traffic Flow Imputation Using Parallel Data and Generative Adversarial Networks, in IEEE Transactions on Intelligent Transportation Systems, pp. 1624–1630, 2019.
[34] X. Chen, Y. Cai, Q. Ye, L. Chen, Z. Li, Graph regularized local self-representation for missing value imputation with applications to on-road traffic sensor data, Neurocomputing 303 (2018) 47–59.
[35] H. Qin, X. Zhan, Y. Li, X. Yang, Y. Zheng, Network-Wide Traffic States Imputation Using Self-Interested Conditional Learning, in ACM SIGKDD Conference on Knowledge Discovery & Data Mining, pp. 1370–1378, 2021.
[36] F. Yu, D. Wei, S. Zhang, Y. Shao, 3D CNN-Based Accurate Prediction for Large-scale Traffic Flow, in International Conference on Intelligent Transportation Engineering, pp. 99–103, 2019.
[37] L. Li, J. Yan, H. Wang, Y. Jin. Anomaly Detection of Time Series with Smoothness-Inducing Sequential Variational Auto-Encoder, in IEEE Transactions on Neural Networks and Learning Systems, pp.1177–1191, 2021.
[38] D. Lao, X. Yang, Q. Wu, J. Yan, Variational Inference for Training Graph Neural Networks in Low-Data Regime through Joint Structure-Label Estimation, in Knowledge Discovery and Data Mining Conference, 2022.
[39] N. Zhao, Z. Li, Y. Li, Improving the Traffic Data Imputation Accuracy Using Temporal and Spatial Information, in International Conference on Intelligent Computation Technology and Automation, 2014.
[40] X. Wu, M. Xu, J. Fang, X. Wu, A multi-attention tensor completion network for spatio-temporal traffic data imputation, IEEE Internet Things J. 9 (20) (2022) 20203–20213.
[41] Y. Chen, Y. Lv, F.-Y. Wang, Traffic flow imputation using parallel data and generative adversarial networks, IEEE Trans. Intell. Transp. Syst. 21 (4) (2020) 1624–1630.
[42] V. A. Le, T. T. Le, P. L. Nguyen, H. T. T. Binh, R. Akerkar, Y. Ji, GCRINT: Network Traffic Imputation Using Graph Convolutional Recurrent Neural Network, in IEEE International Conference on Communications, pp. 1–6, 2021.

Shuo Zhang received the M.S. in computer science and technology in Peking University in 2017. After that, he joint China Aerospace Science and Industry Corporation. Now he is pursing Ph.D. degree in Harbin Institute of Technology Shenzhen with the School of Computer Science and Technology. His research interests include Internet of Things, Traffic Data Mining, Artificial Intelligence, especially algorithms design and industrial system implementation.

Jiayuan Chen received the bachelor's degree in Computer Science in Guilin University of Electronic Technology in 2019. Now he is studying for the master's degree in Harbin Institute of Technology, Shenzhen with the School of Computer Science and Technology. His research interests are in the fields of intelligent transportation systems, especially the field of traffic flow data missing value imputation.

Xiaofei Chen received the bachelor's degree in Computer Science in Dalian Maritime University in 2019. Now he is currently pursuing the M.S. degree from the Harbin Institute of Technology, Shen Zhen China. His current research interests include deep learning for Anomaly detection In Time series data.

Jinyi Chen received the bachelor's degree of civil engineering in Jiangsu University in 2019. Now he is studying for the master's degree in Harbin Institute of Technology, Shenzhen with the School of Computer Science and Technology. His research interests are in the fields of data mining of traffic data.

Hejiao Huang graduated from the City University of Hong Kong and received the PhD degree in computer science in 2004. She is currently a professor in Harbin Institute of Technology, Shenzhen, China, and previously was an invited professor at INRIA, Bordeaux, France. Her research interests include cloud computing, network security, trustworthy computing, Internet of Things, formal methods for system design and wireless networks.